

USER STORIES: TaskManagerMaeconomy

Projectnaam: TaskManagerMaeconomy

Doel van het project: Het ontwikkelen van een mobiele applicatie (React Native) voor taakbeheer met database integratie, uitgevoerd als kennismakingstocht en bewijslast tijdens het loop van de stage.

1. PROJECTOMSCHRIJVING

Dit document bevat de functionele behoeftes en de geprioriteerde takenlijst voor de applicatie “TaskManagerMaeconomy”. De applicatie is in de basis ontworpen om gebruikers in staat te stellen om taken bij te houden binnen een mobiele omgeving. De applicatie maakt gebruik van het React Native framework voor de frontend en maakt een directe koppeling met Firebase voor authenticatie en gegevensopslag.

1.1 Richtlijnen

Het project ontwikkeld onder de volgende richtlijnen:

- **Ontwikkelmethode:** Iteratieve softwareontwikkeling (Agile/Scrum).
- **Architectuur:** Component-gebaseerde opzet met een duidelijke scheiding tussen logica, data (Firebase cloud) en frontend.
- **Codeconventies:** Gebruik van camelCasing voor variabelen en functies, en PascalCasing voor React Native componenten
- **Project overzicht:** Project wordt overzien met een Trello bord.
(<https://trello.com/b/QUYER3Qf/to-do-list-react>)

2. GEBRUIKERSROLLEN

Binnen het project worden de volgende rollen onderscheiden om de scope van de user stories te bepalen:

- **Anonieme Bezoeker (Gast):** Een gebruiker die de applicatie heeft gedownload en opgestart, maar nog niet beschikt over een geverifieerde sessie. Deze actor heeft uitsluitend toegang tot de authenticatieschermen (registratie en login).
- **Geregistreerde Gebruiker:** Een ingelogde user wiens identiteit is gevalideerd via Firebase Authentication. Deze actor heeft volledige toegang tot de CRUD-functionaliteiten van de to-do lijst en het personaliseren van het eigen profiel.

3. Roadmap

De requirements zijn gewogen en geprioriteerd via de MoSCoW-methodiek om een duidelijke roadmap te creëren.

3.1 MUST HAVES

US01: Account Registratie

- **Als** Anonieme Bezoeker
Wil ik een nieuw account kunnen aanmaken met een e-mailadres en een sterk wachtwoord
Zodat mijn persoonlijke to-do items uniek en beveiligd opgeslagen worden.
- **Acceptatiecriteria:**
 - Invoerformulieren valideren of het e-mailadres een geldig formaat bevat (bijv. voorzien van @ en een extensie).
 - Na registratie wordt er een unieke UID gegenereerd binnen Firebase Authentication.

US02: Authenticatie

- **Als** Anonieme Bezoeker
Wil ik veilig kunnen inloggen met mijn geregistreerde e-mailadres en wachtwoord
Zodat ik toegang krijg tot mijn afgeschermd persoonlijke dashboard.
- **Acceptatiecriteria:**
 - Onjuiste combinaties resulteren in een generieke, veilige foutmelding zonder specifieke systeemdata te lekken.
 - Na een succesvolle login wordt de gebruiker direct omgeleid naar het hoofdoverzicht.

US03: Databasekoppeling

- **Als** Geregistreerde Gebruiker
Wil ik dat al mijn wijzigingen direct met een cloud-database synchroniseren
Zodat mijn data niet verloren gaat als ik de applicatie of mijn telefoon afsluit.
- **Acceptatiecriteria:**

- Alle transacties (lezen/schrijven) communiceren rechtstreeks met Firebase Firestore of Realtime Database.
- Data-isolatie is gewaarborgd: to-do documenten zijn strikt gekoppeld aan de unieke UID van de ingelogde gebruiker. Gebruikers kunnen onder geen beding elkaars data opvragen.

US04: Uploaden van Profielfoto

- **Als** Geregistreerde Gebruiker
Wil ik een profielfoto kunnen selecteren vanuit de galerij van mijn toestel en uploaden
Zodat ik mijn accountomgeving kan personaliseren.
- **Acceptatiecriteria:**
 - De geselecteerde afbeelding wordt opgeslagen in een beveiligde map binnen Firebase Storage.
 - De gegenereerde download-URL van de afbeelding wordt opgeslagen in het gebruikersprofiel.
 - De profielfoto wordt na het uploaden direct gecached en correct weergegeven in de interface.

3.2 SHOULD HAVES

US05: To-Do Items Toevoegen

- **Als** Geregistreerde Gebruiker
Wil ik via een invoerveld snel nieuwe taken kunnen toevoegen aan mijn lijst
Zodat ik mijn dagelijkse werkzaamheden direct kan structureren.
- **Acceptatiecriteria:**
 - De interface biedt een tekstveld en een duidelijke actieknop.
 - Lege invoer wordt afgevangen en geeft een foutmelding (validatie).
 - Het toegeegde item verschijnt zonder merkbare haperingen (soepel) direct op het scherm.

US06: Items Bewerken

- **Als** Geregistreerde Gebruiker
Wil ik de tekst of de status van een specifiek item achteraf kunnen wijzigen
Zodat ik typefouten kan corrigeren of de voortgang kan bijwerken.
- **Acceptatiecriteria:**
 - Het selecteren of aanklikken van een taak activeert een bewerkingsmodus waarin de huidige tekst wordt ingeladen.
 - Wijzigingen worden na bevestiging synchroon bijgewerkt in de database en de UI state.

US07: Items Verwijderen

- **Als** Geregistreerde Gebruiker
Wil ik voltooide of overbodige items permanent uit mijn overzicht kunnen wissen
Zodat mijn to-do lijst overzichtelijk blijft.
- **Acceptatiecriteria:**
 - Elk item is voorzien van een intuïtieve delete-actie (zoals een prullenbak-icoon of een swipe-gebaar).
 - Het item wordt direct verwijderd uit de UI-lijst en de bijbehorende database-entiteit wordt gewist.

3.3 COULD HAVES

US08: Sorteren van Items

- **Als** Geregistreerde Gebruiker
Wil ik mijn to-do lijst kunnen sorteren op basis van specifieke criteria (zoals aanmaakdatum of alfabet)
Zodat ik makkelijker prioriteiten kan stellen.
- **Acceptatiecriteria:**
 - De applicatie biedt een sorteermenu of knop.
 - Het sorteren gebeurt lokaal in de frontend state om database-overload te voorkomen.

US09: Filteren van Items

- **Als** Geregistreerde Gebruiker
Wil ik de lijst kunnen filteren op status (bijvoorbeeld 'Openstaand' versus 'Voltooid')
Zodat ik alleen de taken zie die op dat moment relevant zijn.
- **Acceptatiecriteria:**
 - Er zijn tabbladen of filterschakelaars aanwezig in het dashboard.
 - De data wordt dynamisch gefilterd in de component-weergave zonder dat er gegevens uit de database verdwijnen.

4. CODECONVENTIES

de broncode in de gekoppelde GitHub repository

(<https://github.com/Zentyrr/TaskManagerMaeconomy>) is ingericht volgens deze richtlijnen:

- **Versiebeheer:** Functionaliteiten zijn stapsgewijs opgebouwd. Wijzigingen in de Firebase-configuratie en CRUD-componenten zijn traceerbaar via heldere commit-berichten.
- **Naamgeving:** Componenten (bijv. LoginScreen.js, TodoItem.js) hanteren PascalCasing. Interne logica, state-hooks (useState) en database-functies maken gebruik van camelCasing.
- **Architectuur:** Er is strikt gebruikgemaakt van de Firebase SDK om JavaScript functies (async/await) te hanteren bij het ophalen en wegschrijven van de data, wat zorgt voor een stabiele structuur.